Davante

20

BASE DE DATOS

Organización de los tipos de datos objeto

ÍNDICE

/ 1. Introducción y contextualización práctica 4

/ 2. Tipos de datos colección 5

2.1. Declaración de colecciones de objetos 6

/ 3. Tablas de objetos 6

3.1. Concepto 6

3.2. Creación de tablas de objetos 7

/ 4. Tablas con columnas tipo objeto 7

/ 5. Consultas sobre tablas de objetos o tablas con columnas de tipo objeto 8

/ 6. Inserción de objetos en una tabla 9

/ 7. Modificación de objetos en una tabla 10

/ 8. Eliminación de objetos de una tabla 11

/ 9. Cláusula VALUE en consultas 12

/ 10. Cláusula REF en consultas 13

/ 11. Caso práctico 1: “Declaración de tipos de colecciones” 14

ÍNDICE

/ 12. Caso práctico 2: “Declaración de tipo de objeto y creación de tablas” 14

/ 13. Resumen y resolución del caso práctico de la unidad 15

/ 14. Bibliografía 16

© MEDAC ISBN: 978-84-19885-68-5
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

OBJETIVOS

Estudiar las características fundamentales y la gestión de los diferentes tipos de datos colección.

Dominar las propiedades y funcionalidades que proporcionan las tablas de objetos y aquellas con columnas de tipo objeto en una BD objeto-relacional.

Analizar cómo realizar diversas consultas sobre tablas que almacenan datos de tipo objeto.

Entender el proceso para insertar, modificar o eliminar objetos en una tabla que contiene datos de tipo objeto.

/ 1. Introducción y contextualización práctica

El estudio comienza con el análisis de los tipos de datos colección. Los conceptos abordados en este tema, como *arrays* y tablas anidadas, ya fueron introducidos en temas anteriores, adaptados a los registros en PL/SQL. En este caso, la idea se aplica específicamente a los tipos de colecciones, lo que facilitará su comprensión.

Igual que sucede con otros tipos de datos, como VARCHAR o NUMBER, los tipos de datos objeto pueden organizarse en tablas, ya sea en su totalidad o solo en algunas de sus columnas. Esta es una característica clave de las BBDD objeto-relacionales, que exploraremos con mayor profundidad en este tema. Nos familiarizaremos con su concepto y aprenderemos a crearlas.

Además, estas tablas permiten realizar consultas SQL sobre los datos mediante la instrucción SELECT. También es posible emplear otras sentencias SQL, previamente estudiadas en la asignatura, para insertar, modificar o eliminar datos en tablas que contienen tipos de datos objeto.

	COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
1	ID_PERSONA	VARCHAR2(9 BYTE)	Yes	(null)	1	(null)
2	UNALUMNO	ALUMNO	Yes	(null)	2	(null)
3	UNPROFESOR	PROFESOR	Yes	(null)	3	(null)

Fig.1. Imagen de tabla con columnas tipo objeto.

A continuación, se presenta un caso práctico a través del cual podremos aproximarnos de forma aplicada a la teoría de este tema.

Escucha el siguiente audio donde planteamos la contextualización práctica de este tema. Encontrarás su resolución en el apartado 'Resumen y resolución del caso práctico'.

Audio Intro. "Tablas con datos de tipo objeto"

https://on.soundcloud.com/ws4hiRMcXboVyjXT6

/ 2. Tipos de datos colección

Una colección es un conjunto de elementos del mismo tipo, que poseen características comunes. Este concepto puede compararse con el de matriz o vector en la programación orientada a objetos.

Los tipos de datos colección **permiten representar relaciones 1:N** en Oracle. Como se mencionó, estos tipos de datos están formados por varios elementos del mismo tipo. Además, vienen con **métodos constructores predefinidos que facilitan la creación de objetos colección**, similar a lo que ocurre con los tipos de datos objeto.

En Oracle, existen **dos tipos principales de datos colección**, los cuales veremos en la siguiente tabla:

Tipo de datos colección	Características
VARRAY	Es un conjunto de elementos del mismo tipo, organizados mediante un índice. Tiene una longitud variable, aunque es necesario especificar un límite máximo al declararlo. Se utiliza comúnmente cuando se conoce el número de elementos y estos pueden ser ordenados.
TABLA ANIDADA (Nested Table)	Es un conjunto de elementos del mismo tipo, sin un orden específico. No tiene tamaño fijo predefinido. Es preferible cuando se requiere insertar, modificar o eliminar grandes cantidades de datos, o cuando se trabaja con un número indeterminado de elementos.

Tabla 1. Características de los tipos de datos colección.

Investigamos...

Busca en Google Scholar 'uso de colecciones en PL/SQL' para explorar cómo las colecciones de objetos pueden optimizar la gestión de datos complejos en bases de datos objeto-relacionales.

Los tipos de datos de colección **pueden albergar cualquier tipo de objeto que se haya declarado previamente**. De hecho, una tabla en una base de datos puede incluir columnas que sean de tipos de datos de colección, permitiendo así realizar operaciones sobre estas columnas de la misma forma que con tipos de datos convencionales como VARCHAR o NUMBER.

En cuanto al acceso a los tipos de colección VARRAY desde SQL, este está limitado, ya que no es posible aplicar condiciones de consulta sobre los elementos individuales. Es decir, solo se puede recuperar el VARRAY completo como un conjunto. Sin embargo, **en PL/SQL, es posible implementar bucles que permitan manejar y recorrer internamente un VARRAY**, de manera similar a como se haría en cualquier lenguaje de programación.

Audio 1. "Usos de VARRAY y NESTED TABLE"

https://on.soundcloud.com/zm19nnUoEXxK3kEz5

2.1. Declaración de colecciones de objetos

La declaración de tipos de colecciones VARRAY y tablas anidadas sigue la misma sintaxis que se estudió en temas anteriores de la asignatura en PL/SQL. Sin embargo, si se crea directamente **desde SQL**, es necesario incluir la sentencia **CREATE OR REPLACE**. En la figura 2 se muestra un ejemplo.

```
TYPE nombr_tipo IS VARRAY (máx_capacidad) OF tipo_elemento;
TYPE nombr_tipo IS TABLE OF tipo_elemento;
```

Fig.2. Sintaxis de declaración de colecciones.

Al igual que sucede con los tipos de objeto, es esencial **inicializar las colecciones**; de lo contrario, su valor será NULL. Como se mencionó anteriormente, Oracle genera constructores para los tipos de colecciones, de manera similar a como lo hace para los tipos de objeto, permitiendo **pasar los valores de la colección como parámetros**. En la figura 3, se muestra un ejemplo.

```
DECLARE
    TYPE Departamentos IS VARRAY (5) OF VARCHAR2(20);
    Var_nombres Departamentos;
BEGIN
    Var_nombres := Departamentos ('Mates', Lengua', 'Inglés',
                                  'Tecnología', 'Sociales')
```

Fig.3. Sintaxis de declaración de array.

En lugar de usar **VARRAY**, también podríamos haber **utilizado tablas anidadas** con la expresión de la figura 4.

```
TYPE Departamentos IS TABLE OF VARCHAR2(20);
```

Fig.4. Sintaxis de declaración de tabla anidada.

Para **acceder a los elementos de la colección o modificar sus valores**, se puede proceder como se muestra en la figura 5.

```
Var_nombres(2) := 'Historia';
Dbms_output.put_line(Var_nombres(2));
```

Fig.5. Sintaxis de acceso a la colección

En el siguiente vídeo se profundiza en estos conceptos mediante otros ejemplos:

Vídeo 1. "Declaración de colecciones en Oracle"

https://bit.ly/2XH1CCZ

3. Tablas de objetos

A continuación, exploraremos el concepto y la creación de tablas de objetos, esenciales para gestionar datos de tipo objeto en bases de datos.

3.1. Concepto

Para aumentar la utilidad de los objetos dentro de un entorno de bases de datos, es fundamental que **puedan organizarse y almacenarse en forma de tabla**. De este modo, pueden manejarse como cualquier otro tipo de dato que hemos explorado a lo largo del curso.

Es posible **crear tablas compuestas exclusivamente por objetos de un mismo tipo, o combinar estos objetos con otros tipos de datos en columnas específicas**.

3.2. Creación de tablas de objetos

Una vez definidos los tipos de objetos, junto con sus atributos y métodos, podemos declarar **una tabla** compuesta por esos objetos. Para ello, se utiliza la sentencia **CREATE TABLE**, seguida del nombre de la tabla que se desea crear y del tipo de objeto específico. Se muestra un ejemplo en la figura 6.

```
CREATE TABLE nombre_tabla OF tipo_objeto;
```

Fig.6. Sintaxis para declarar tablas de objetos.

Esta opción es **adecuada cuando la tabla va a estar compuesta únicamente por objetos de ese tipo**.

Es importante tener en cuenta que, **una vez creada la tabla, no se podrá modificar el tipo de objeto definido**, es decir, no será posible redefinirlo.

Por ejemplo, para crear una tabla denominada 't_profes' compuesta por objetos del tipo 'profesor', se utilizaría la sentencia de la figura 7.

```
CREATE TABLE t_profes OF profesor;
```

Fig.7. Ejemplo de creación de tabla de objetos 'profesor'.

Una vez que los objetos se hayan almacenado en la tabla, permanecerán allí hasta que sean eliminados explícitamente. Esta permanencia contrasta con los objetos almacenados como variables, los cuales desaparecen una vez finalizada su ejecución. Sin embargo, cuando se almacenan en una tabla, su existencia es continua.

Por tanto, es importante comprender que **el tipo de objeto definido puede utilizarse en cualquier momento**, ya sea para crear objetos que se almacenarán en una tabla o para cualquier otro propósito que el usuario decida, como su uso como variable o parámetro.

Casos de éxito...

Oracle ha implementado tablas de objetos en sistemas críticos, permitiendo un manejo eficiente de datos complejos, como los sistemas de reservas de aerolíneas. ¡Un claro ejemplo de la teoría aplicada con éxito!

/ 4. Tablas con columnas tipo objeto

Como se mencionó anteriormente, cualquier tipo de objeto puede ser integrado en una tabla, ya sea como el contenido completo de la misma o como una de sus columnas.

Por consiguiente, es posible manipular la tabla empleando las **sentencias SQL y PL/SQL** estudiadas durante el curso, tanto para la **inserción de elementos como para consultar sus atributos o modificar los datos almacenados**.

El proceso de **crear una tabla con columnas de tipo objeto es similar al de cualquier otro tipo de dato**, como VARCHAR2, NUMBER o INTEGER. Es necesario **especificar el objeto en la declaración de la tabla**. Por ejemplo, si deseamos crear una tabla para registrar información sobre las asignaturas en un centro educativo, podemos incluir una columna con un tipo objeto denominado 'profesor', como se muestra en la figura 8.

```
CREATE TABLE prueba_asignaturas (
    Id_asignatura NUMBER(4),
    Profesor1 profesor
) ;
```

Fig.8. Creación de tabla con columna tipo objeto 'profesor'.

En la figura 9 se muestra un ejemplo implementado en Oracle SQL Developer en el que se crea una tabla denominada PERSONAS, en la que se identifican las siguientes tres columnas:

- **Id_persona**: Se refiere a la identificación de la persona, y su contenido podría ser, por ejemplo, el DNI.
- **unAlumno**: Columna que almacena objetos del tipo de datos ALUMNO, previamente declarado.
- **unProfesor**: Columna que almacena objetos del tipo de datos PROFESOR, que se declaró anteriormente.

```
CREATE TABLE PERSONAS (
    id_persona VARCHAR2(9),
    unAlumno ALUMNO,
    unProfesor PROFESOR
);
```

Fig.9. Creación de tabla con columnas tipo objeto.

Tras crear la tabla, se pueden ver sus propiedades y, en la sección de Columnas, se presentan los datos de la figura 10.

	COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
1	ID_PERSONA	VARCHAR2(9 BYTE)	Yes	(null)	1	(null)
2	UNALUMNO	ALUMNO	Yes	(null)	2	(null)
3	UNPROFESOR	PROFESOR	Yes	(null)	3	(null)

Fig.10. Tabla con columnas tipo objeto.

5. Consultas sobre tablas de objetos o tablas con columnas de tipo objeto

Como hemos visto en temas anteriores de esta asignatura, la sentencia **SELECT** nos permite **realizar** una variedad de **consultas en las tablas de una base de datos**. De manera similar a lo aprendido hasta este punto, también se puede utilizar esta sentencia para efectuar **consultas en tablas compuestas completamente por objetos** o en aquellas tablas que contienen **columnas de tipo objeto**.

Como se puede ver en la figura 11, es posible acceder a la información de la tabla mediante la sentencia mostrada, incluyendo el tipo de objeto 'profesor', de la misma forma que se haría con cualquier otro tipo de datos.

```
SELECT * FROM prueba_asignaturas ;
```

Fig.11. Consulta sobre tabla con columnas tipo objeto.

La forma de realizar la consulta no cambia, tanto si la tabla está compuesta por objetos como si tiene columnas de tipo objeto.

Al igual que con otros tipos de datos, **se pueden emplear todas las cláusulas** que hemos visto en temas anteriores junto a la sentencia **SELECT**, destacando algunas como **WHERE, ORDER BY, JOIN, SUM, y MAX**.

En el siguiente vídeo se muestra, de manera práctica, cómo realizar consultas sobre una tabla con columnas tipo objeto, y cómo las resuelve el sistema Oracle Express.

Vídeo 2. "Consulta sobre tabla con columnas tipo objeto"

https://bit.ly/39umdQw

En bases de datos que manejan tipos de objeto, existe el concepto de **alias**, utilizado **para nombrar las tablas**. El alias facilita el acceso al contenido de la tabla. En la figura 12 se muestra cómo se utiliza.

```
CREATE TYPE profesor AS OBJECT (nombre VARCHAR(2));
CREATE TABLE profe1 OF profesor;
CREATE TABLE profe2 (c1 profesor);
```

Fig.12. Ejemplo de creación de tablas.

La primera tabla, 'profe', almacena objetos de tipo 'profesor', mientras que 'profe2' incluye una columna de tipo 'profesor'. Se accede a la información como se muestra en la figura 13.

```
SELECT nombre AS OBJECT profe1;
SELECT p.c1.nombre FROM profe2 p;
```

Fig.13. Ejemplo de consultas a tablas.

En la primera consulta, 'nombre' actúa como una columna en la tabla 'profe1', dado que los atributos de los objetos se reflejan como columnas en la tabla. En la segunda consulta, se debe especificar que 'nombre' es un atributo del objeto tipo 'profesor' dentro de la columna 'c1', lo cual exige el uso de un alias, ya que no está permitido emplear directamente la expresión 'profe2.c1.nombre'.

/ 6. Inserción de objetos en una tabla

Otra tarea esencial en una base de datos es la inserción de campos en una tabla. Así como se pueden insertar campos con tipos de datos comunes (tema que ya hemos estudiado en la asignatura), también **es posible insertar objetos**. El comando utilizado para esta operación, independientemente del tipo de datos u objeto, es el mismo: **INSERT**.

La diferencia radica en que, a**l insertar una fila o un campo de un tipo de objeto específico, es necesario indicar a qué tipo concreto pertenece**.

En la figura 14, se muestra cómo se **declaran dos instancias del objeto** 'profesor', se inicializan y luego se añaden a la tabla 'profe1'.

```
DECLARE
    P1 profesor;
    P2 profesor;
BEGIN
    P1 := NEW profesor ('1234', 'Ana',
                        'López Rey', 1500, 'Matemáticas');
    P2 := NEW profesor('1235', 'Marcos',
                        'Fdez Ruiz',1300, 'Historia');
    INSERT INTO profe1 VALUES (P1);
    INSERT INTO profe1 VALUES (P2);
END ;
```

Fig.14. Ejemplo de inserción de objetos en tabla.

En el siguiente recurso podrás aprender más sobre el apartado anterior.

Investigamos...

Investiga sobre 'integridad referencial en bases de datos objeto-relacionales' en Google Scholar para entender cómo las bases de datos manejan las relaciones entre objetos.

Como se puede ver en la imagen anterior, se realiza una **inserción de objetos en una tabla compuesta exclusivamente por ellos**. El procedimiento para insertar en una tabla que contiene columnas de tipo objeto sería similar.

Supongamos que tenemos la tabla 'profe2', la cual cuenta con una columna de tipo objeto 'profesor', además de otras columnas con tipos de datos comunes. Para llevar a cabo la **inserción de una fila en esta tabla**, utilizaríamos el **comando** mostrado en la figura 15.

```
INSERT INTO profe2 VALUES (P1,'111111A', 35);
```

Fig.15. Ejemplo de inserción de una fila en tabla con columnas objeto.

El procedimiento es igual que con un tipo de dato habitual. **Podemos instanciar un objeto previamente o inicializarlo directamente en la sentencia INSERT**, como se muestra en la figura 16.

```
INSERT INTO profe2 VALUES (
    ('1234', 'Ana', 'López Rey', 1500, 'Matemáticas'),
    '111111A', 35);
```

Fig.16. Ejemplo de inserción de objetos en tabla con columnas objeto.

/ 7. Modificación de objetos en una tabla

Al igual que vimos en los puntos anteriores sobre el uso de sentencias SQL con objetos, tanto para consultas como para insertar datos en una tabla, también se puede utilizar la **sentencia UPDATE**, que ya estudiamos, **para modificar el valor de ciertos campos, incluso si son de tipo objeto**.

Sin embargo, la sentencia no se construye de la misma manera cuando trabajamos con una tabla de objetos. La **sintaxis** que se debe emplear se muestra en la figura 17.

```
UPDATE ejemplo_tabla SET ejemplo_tabla.atributo = valor
WHERE ejemplo_tabla.atributoBusca = valorBusca;
```

Fig.17. Sintaxis de modificación en una tabla de objetos.

Para entenderlo mejor, vamos a aclararlo con un ejemplo: Supongamos que queremos modificar el nombre del objeto de tipo profesor 'P1' (id_profe = 1234), que hemos incluido en la tabla de objetos profesor 'Profe1', como se muestra en la figura 18.

```
UPDATE profe1 p SET p.nombre = 'Maria'
WHERE p.id_profe = '1234';
```

Fig.18. Ejemplo de modificación de tabla de objetos.

La sentencia de la imagen anterior busca, en la tabla 'profe1', el objeto cuyo id_profe sea '1234' (correspondiente al objeto 'P1') y modifica su atributo nombre.

En caso de utilizar una **tabla** que contiene **columnas de tipo objeto**, es necesario **especificar la columna que contiene dichos objetos**, como se muestra en la figura 19.

```
UPDATE ejemplo_tabla
SET ejemplo_tabla.columna.atributo = valor
WHERE ejemplo_tabla.columna.atributoBusca = valorBusca;
```

Fig.19. Sintaxis de modificación de tabla con columnas tipo objeto.

En la figura 20, se muestra un ejemplo de cómo actualizar los datos en una tabla (profe2) con una columna de tipo objeto (c1), utilizada en puntos anteriores del tema para ilustrar este concepto.

```
UPDATE profe2 p SET p.c1.nombre = 'Maria'
WHERE p.c1.id_profe = '1234';
```

Fig.20. Ejemplo de modificación de tabla con columnas tipo objeto.

En ambos casos, ya sea una **tabla de objetos o con columnas de tipo objeto**, se puede **reemplazar un objeto completo por otro**, incluyendo todos sus campos. La diferencia es que, en las tablas con columnas tipo objeto, hay que referirse a la columna que lo contiene.

/ 8. Eliminación de objetos de una tabla

Después de haber abordado la inserción y modificación de objetos almacenados en tablas de una base de datos, es momento de estudiar cómo se realiza la eliminación de estos objetos, siguiendo un proceso similar al de las sentencias SQL de tipo DML que hemos visto a lo largo del curso.

La eliminación de objetos en tablas se lleva a cabo utilizando la sentencia **DELETE**, la misma que se emplea comúnmente en SQL para cualquier otro tipo de dato. Al igual que en casos anteriores, esta sentencia **puede combinarse con consultas SELECT** si se desea borrar objetos que cumplan con ciertos criterios definidos por una consulta. En definitiva, los principios estudiados en temas anteriores sobre comandos DML también se aplican aquí a tablas que contienen tipos de datos objeto.

La **sintaxis** es prácticamente idéntica a la que hemos estudiado previamente, como se puede apreciar en la figura 21.

```
DELETE FROM ejemplo_tabla
WHERE condición;
```

Fig.21. Sintaxis para la eliminación de objetos en una tabla.

La cláusula **WHERE** es **opcional**, permite **establecer una condición para el borrado de elementos**. Si no se especifica una condición, todos los elementos de la tabla serán eliminados.

Por ejemplo, para eliminar de la tabla 'profe1' el objeto de tipo 'profesor' cuyo id sea '1234', se utilizaría la sentencia de la figura 22.

```
DELETE FROM profe2 p
WHERE p.c1.id_profe = '1234';
```

Fig.22. Ejemplo de eliminación de objetos de una tabla.

El ejemplo anterior se aplica a una tabla compuesta exclusivamente por objetos. Sin embargo, **en tablas que contienen columnas de tipo objeto** junto **con otras de diferentes tipos de datos**, la sentencia **DELETE** se formularía de manera ligeramente distinta. Al igual que en el caso de la modificación de objetos, aquí también **es necesario especificar la columna que contiene los datos de tipo objeto**.

Consideremos el ejemplo de la figura 23, para la tabla 'profe2', donde la columna 'c1' almacena objetos de tipo 'profesor'.

```
DELETE FROM profe2 p
WHERE p.c1.id_profe = '1234';
```

Fig.23. Ejemplo de borrado de objetos de una tabla con columnas tipo objeto.

/ 9. Cláusula VALUE en consultas

La cláusula **VALUE se utiliza junto con consultas SELECT**, y es especialmente **útil cuando se necesita hacer referencia al objeto completo** en lugar de solo a uno de sus atributos.

Por ejemplo, si queremos **seleccionar todos los objetos que cumplen con una cierta condición**, podríamos emplear la expresión de la figura 24.

```
SELECT VALUE(profe1)
FROM profe1 WHERE salario >  500;
```

Fig.24. Ejemplo de uso de VALUE en consultas.

Es posible realizar la misma consulta **utilizando alias**, o incluso **combinándola con otras sentencias**, **como** las de **inserción**. En la figura 25 se muestra un ejemplo de ello.

```
SELECT VALUE(p)
FROM profe1 p WHERE p.salario >  500;
```

Fig.25. Ejemplo de uso de VALUE con alias en consultas.

En el ejemplo de la figura 26, los objetos de tipo 'profesor' que cumplen una condición específica se insertan en una tabla llamada 'profes3', que también contiene objetos de ese tipo.

```
INSERT INTO profes3 SELECT VALUE(p)
FROM profe1 p WHERE p.salario >  500;
```

Fig.26. Ejemplo de inserción de objetos en una tabla de objetos usando VALUE.

Además, la cláusula **VALUE puede emplearse para realizar comparaciones entre objetos**, como se muestra en la figura 27.

```
SELECT p1.salario FROM profe1 p
JOIN profes3 p2 ON VALUE(p1) = VALUE(p2);
```

Fig.27. Ejemplo de comparación de objetos usando VALUE.

También se suele **combinar con la cláusula INTO**. Esta combinación **permite almacenar el resultado de una consulta en una variable del mismo tipo de objeto**, lo que facilita su manipulación posterior accediendo a sus atributos, de manera similar a cualquier otro objeto. En el ejemplo de la figura 28, se muestran dos objetos de tipo 'profesor' llamados 'obj_profe' y 'obj2_profe'. El resultado de la consulta se almacena en el primero y luego se asigna al segundo.

```
SELECT VALUE(p) INTO obj_profe
FROM profe1 p WHERE p.id_profe = '1234';
Obj2_profe := obj_profe;
```

Fig.28. Ejemplo de utilización de las cláusulas VALUE e INTO.

/ 10. Cláusula REF en consultas

La cláusula **REF** se utiliza para obtener una **referencia a un objeto** en lugar de extraer el objeto completo. Esta función es particularmente útil cuando se desea **crear relaciones entre tablas utilizando referencias a objetos**, en lugar de replicar toda la información de los objetos en cada tabla. Este enfoque ayuda a **mantener la integridad de los datos y evita la redundancia**.

Enlaces de interés...

En el siguiente enlace encontrarás la documentación oficial de Oracle sobre la cláusula REF. Revísala para ampliar tus conocimientos sobre esta cláusula: https://docs.oracle.com/en/database/oracle/oracle-database/19/sqlrf/REF.html

Por ejemplo, en una base de datos que gestiona cursos y profesores, podríamos tener una **tabla** llamada '**cursos**' que incluye una columna 'profesor_ref', donde se almacenan **referencias a los objetos de tipo Profesor**. Así, la tabla 'cursos' no necesita guardar todos los detalles del profesor, sino solo una referencia al objeto existente en la tabla 'profesores'. Esto se logra con la **cláusula REF** en la sentencia SQL, como se muestra en la figura 29.

```
CREATE TABLE cursos (
    id NUMBER,
    nombre_curso VARCHAR2(50),
    profesor_ref REF Profesor
);
```

Fig.29. Creación de tabla con referencia a profesor.

Para insertar una fila en la tabla 'cursos' y vincularla con un profesor específico, podríamos usar la cláusula **REF** como se muestra en la figura 30.

```
INSERT INTO cursos (id, nombre_curso, profesor_ref)
SELECT 1, 'Matemáticas', REF(p)
FROM profesores p
WHERE p.nombre = 'Sofía';
```

Fig.30. Inserción con referencia de objeto.

En este ejemplo, el comando INSERT INTO cursos utiliza la cláusula **REF** para crear **una referencia al objeto Profesor** cuyo nombre es 'Sofía'. Esta referencia se almacena en la columna 'profesor_ref' de la tabla 'cursos'. Así, cualquier **modificación** en los datos del profesor **en la tabla 'profesores' se refleja automáticamente en la tabla 'cursos'**, lo que **mantiene la consistencia e integridad de los datos** y **simplifica las actualizaciones**.

Recuerda...

La cláusula VALUE se utiliza para recuperar y manipular un objeto completo desde una tabla, mientras que REF obtiene una referencia a un objeto dentro de una tabla. Ambas son esenciales para gestionar eficientemente datos en bases de datos Oracle.

/ 11. Caso práctico 1: "Declaración de tipos de colecciones"

Planteamiento: Pondremos en práctica los conceptos teóricos que hemos aprendido previamente, para lo cual declararemos varios tipos de colecciones.

Nudo: Es necesario definir un objeto básico llamado 'cliente', que incluirá los atributos id_cliente (NUMBER), nombre (VARCHAR) y teléfono, el cual será un VARRAY de 10 elementos de tipo VARCHAR2.

Desenlace:

```
CREATE TYPE lista_telefono
    AS VARRAY (10) OF VARCHAR2(20);
```

Fig.31. Creación de colección.

```
CREATE TYPE cliente AS OBJECT(
    Id_cliente NUMBER,
    Nombre VARCHAR2(200),
    teléfono lista_telefono
);
```

Fig.32. Creación de tipo objeto con campo de tipo lista_telefono.

Primero, definiremos el array de objetos y, a continuación, procederemos a declarar el tipo de objeto cliente como se muestra en las figuras 31 y 32.

/ 12. Caso práctico 2: "Declaración de tipo de objeto y creación de tablas"

Planteamiento: Pongamos en práctica la teoría estudiada en los últimos apartados del tema. Comenzaremos declarando un tipo de objeto y creando dos tablas: una compuesta completamente por objetos de ese tipo y otra con solo una columna. Luego, insertaremos valores en ambos objetos utilizando los comandos previamente aprendidos. Finalmente, realizaremos consultas sobre las tablas creadas.

Nudo: Es recomendable apoyarse en los ejemplos prácticos analizados a lo largo del tema, o incluso en los abordados en temas anteriores. Esto te ayudará a reforzar la comprensión y aplicación de los conceptos.

Desenlace:

```
CREATE TYPE profesor AS OBJECT (
    id NUMBER,
    nombre VARCHAR2(50),
    apellidos VARCHAR2(100),
    salario NUMBER,
    asignatura VARCHAR2(50)
);

CREATE TABLE profe1 OF profesor;
CREATE TABLE profe2 (
    c1 profesor
);

DECLARE
    P1 profesor;
    P2 profesor;
BEGIN
    P1 := profesor(1234, 'Sofía', 'Valencia Acha', 1500, 'Matemáticas');
    P2 := profesor(1235, 'Esther', 'Fdez Ruiz', 1300, 'Historia');

    INSERT INTO profe1 VALUES (P1);
    INSERT INTO profe1 VALUES (P2);
    INSERT INTO profe2 (c1) VALUES (P1);

    SELECT * FROM profe1;
    SELECT p.c1.nombre FROM profe2 p;
END;
```

Fig.33. Posible solución al caso práctico planteado.

En este caso, el código de ejemplo mostrado en los puntos anteriores podría ser válido. Cabe destacar que el tipo de datos profesor fue definido en apartados anteriores, lo que facilita su reutilización en este contexto.

/ 13. Resumen y resolución del caso práctico de la unidad

En este tema, hemos explorado los tipos de datos colección para manejar grupos de objetos en nuestros *scripts*. Además, hemos aprendido a crear tablas para almacenar estos objetos, ya sea como elementos únicos en la tabla o dentro de una columna específica.

Finalmente, hemos abordado cómo realizar las diferentes operaciones DML para la consulta, inserción, modificación y eliminación de objetos en la base de datos. También hemos visto cómo utilizar las cláusulas VALUE y REF, que nos permiten manejar la información de manera eficaz.

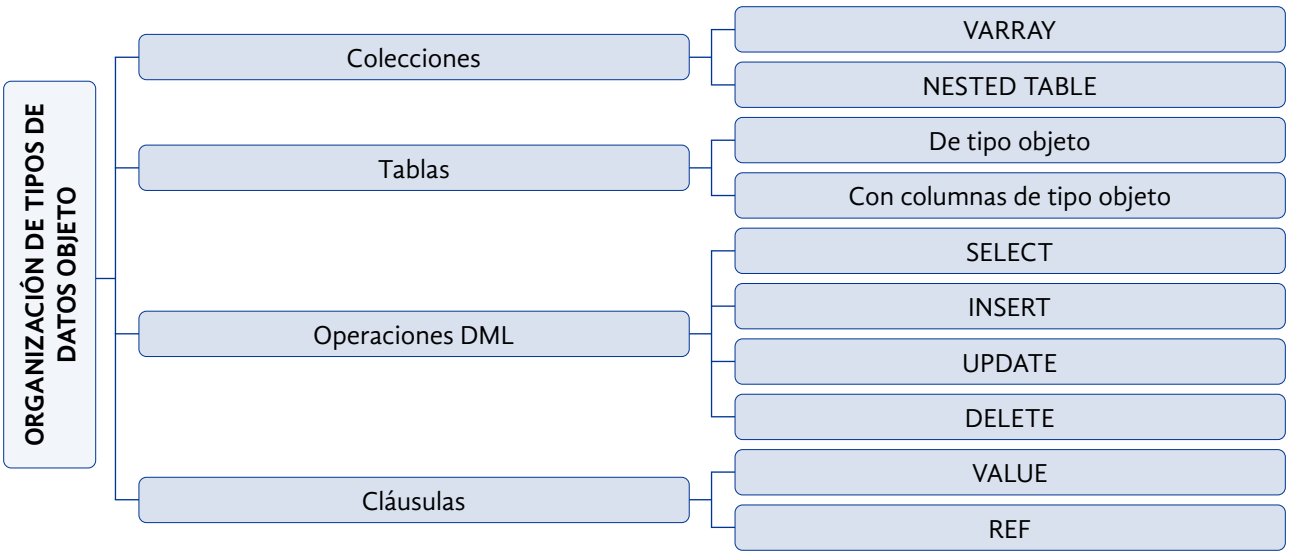


Fig.34. Esquema resumen del tema.

Resolución del caso práctico de la unidad

A lo largo del tema, hemos visto que existen, principalmente, dos tipos de tablas que pueden contener tipos de dato objeto: aquellas que están compuestas enteramente por objetos de un solo tipo y aquellas que combinan columnas con tipos de datos convencionales (como VARCHAR2, NUMBER, INTEGER, etc.) junto con otras columnas que contienen objetos de un mismo tipo. Este aspecto se establece durante la creación de la tabla, de manera similar a lo que ocurre al definir tablas que contienen otros tipos de datos, como se ha abordado en temas anteriores de la asignatura.

Del mismo modo, si almacenamos información sobre objetos en formato de tabla, es fundamental que dicha información sea accesible mediante las herramientas disponibles, como SQL. Esto permite consultar, modificar, añadir o eliminar datos, incluso cuando se trata de objetos. De hecho, como hemos analizado en esta unidad didáctica, el proceso para realizar estas operaciones es prácticamente idéntico al que aprendimos en temas previos del curso, donde trabajamos con tablas que contienen otros tipos de datos.

/ 14. Bibliografía

Connolly, T. y Begg, C. (2005). *Sistemas de bases de datos* (5.ª ed.). Madrid, España. Addison-Wesley.

Elmasri, R. y Navathe, S. (2007). *Fundamentos de sistemas de bases de datos*. Addison-Wesley.

López Montalbán, I. y De Castro, M. (2014). *Gestión de bases de datos: Técnico Superior en Administración de Sistemas Informáticos en Red*. Madrid. Garceta D.L.

Oppel, A. (2009). *Databases. A Beginner's Guide*. McGraw-Hill Education.

Pérez Marqués, M. (2016). *Administración básica de bases de datos con Oracle 12c SQL*. Madrid, España. Alfaomega.

Sánchez, G. C. (2001). *Sistemas gestores de bases de datos*. Paraninfo.

Vela Sánchez, B. (2015): *Diseño de bases de datos objeto-relacionales con UML*. Madrid, España. Dykinson.